Project of Project Based Learning II

A Transformer-Based Classification: A Case Study of Semantic Context Evaluation for Mobile Spam Message Detection

Project Documentation



Institution

College of Global AI Convergence
Seoul Christian University, Korea

Authors

Chaulagai Shishir (20240301022)
Bishwokarma Rajkumar (20240301097)
Mehta Avishek (20240301059)

Contents

Abstract 3

Literature Review 4

1 Introduction 6

2 Dataset Description 7

3 Exploratory Data Analysis 8

4 Data Preprocessing 10

5 Methodology 12

6 Model Architectures 14

7 Experimental Setup 16

8 Results and Performance Analysis 17

9 Semantic Context Evaluation 19

10 Discussion 21

11 Conclusion 22

References 23

Appendix A: Hyperparameter Configuration 24

Appendix B: Sample Predictions 24

Abstract

Mobile SMS spam and phishing (smishing) attacks pose significant cybersecurity threats to billions of smartphone users worldwide. Traditional keyword-based and shallow machine learning approaches struggle to detect spam that relies on obfuscation and semantic manipulation. This study presents a comparative evaluation of three classification approaches for mobile spam detection: TF-IDF with Logistic Regression, an embedding-based Bidirectional LSTM (BiLSTM), and a fine-tuned Transformer model (DistilBERT).

Using the UCI SMS Spam Collection of 5,572 labeled messages, the fine-tuned DistilBERT model achieved 99.28% accuracy, 97.32% precision, 97.32% recall, and a 99.90% ROC-AUC on a held-out test set of 836 messages. Compared with the TF-IDF baseline, it reduced the number of missed spam messages (false negatives) from 18 to 3, an 83% reduction. A qualitative analysis of 19 messages that the TF-IDF baseline misclassified and DistilBERT classified correctly shows that the Transformer is better at detecting obfuscated phishing, social engineering, and context-dependent spam signals while keeping false positives low. These findings provide empirical support for deploying Transformer-based semantic analysis in mobile spam filtering systems.

Keywords: SMS spam detection, Transformer models, DistilBERT, natural language processing, semantic context analysis, mobile security, phishing detection.

Literature Review

Evolution of Spam Detection Approaches

Spam detection has progressed through four broad generations of techniques, each addressing the limitations of its predecessors while introducing new challenges.

Rule-based systems (1990s to early 2000s). Early filters relied on manually written rules that matched keywords (for example "FREE", "WINNER", "CLICK HERE") or surface patterns such as excessive capitalization and repeated exclamation marks. These systems were interpretable and needed no training data, but they demanded constant maintenance, were easily bypassed through character substitution, generalized poorly to new spam variants, and produced false positives on legitimate messages containing trigger words.

Statistical machine learning (2000s to 2010s). Data-driven classifiers such as Naive Bayes, Support Vector Machines (SVMs), and TF-IDF with Logistic Regression learned decision rules from labeled examples. Naive Bayes assumes word independence and is effective for simple spam but weak on context-dependent messages. SVMs achieve better class separation but depend on careful feature engineering. TF-IDF with Logistic Regression forms a strong and widely used baseline, yet it is limited to unigram or bigram features and has no notion of semantics.

Deep learning (2010s to present). Recurrent Neural Networks (RNNs) process text sequentially and capture word order, but they suffer from vanishing gradients. Long Short-Term Memory (LSTM) networks (Hochreiter & Schmidhuber, 1997) use gating to preserve information over longer spans, and Convolutional Neural Networks (CNNs) detect local n-gram patterns. These models improved on statistical methods but still struggle with very long-range dependencies and typically need more labeled data.

Transformer-based models (2017 to present). The Transformer architecture (Vaswani et al., 2017) replaced recurrence with self-attention, which computes the relevance between every pair of tokens regardless of distance. Combined with pretraining on large unlabeled corpora (BERT, GPT, RoBERTa), it yields contextual embeddings in which a word's representation depends on its surrounding text. These models set new standards on language understanding tasks, which motivates their application to spam detection.

Transformer Models for Text Classification

BERT. Devlin et al. (2019) introduced Bidirectional Encoder Representations from Transformers (BERT), pretrained with masked language modeling and next-sentence prediction on large text corpora. Its bidirectional context makes it well suited to detecting subtle semantic cues, such as the indirect wording of a phishing message.

DistilBERT. Sanh et al. (2019) used knowledge distillation to create DistilBERT, which is about 40% smaller than BERT-base (66 million versus 110 million parameters), about 60% faster at inference, and retains about 97% of BERT's language understanding performance. This efficiency makes it an attractive candidate for latency-sensitive and resource-constrained deployments such as mobile messaging.

SMS Spam Detection Research

- Almeida et al. (2011) created the UCI SMS Spam Collection and evaluated Naive Bayes, SVM, and other classifiers on it, establishing the benchmark used in most later work.
- Gupta et al. (2018) combined TF-IDF features with ensemble learning and reported accuracy of roughly 96%, but with a comparatively high rate of missed spam.
- Roy et al. (2020) applied CNN and LSTM architectures to SMS filtering and reported accuracy of about 98%, with limited analysis of contextual understanding.
- Studies that evaluate pretrained Transformers specifically on SMS spam remain comparatively scarce, as much of the Transformer literature targets email or social media text.

Research Gap and Motivation

Despite advances in natural language processing, SMS spam detection remains underexplored in Transformer research. Four gaps motivate this work:

1. Few studies compare Transformers rigorously against traditional baselines on identical SMS data and splits.
2. Few studies explain, through error analysis, why Transformers outperform keyword-based approaches on particular message types.
3. Practical evidence on the feasibility of real-time inference for mobile or carrier deployment is limited.
4. Lightweight Transformer variants such as DistilBERT have received little attention for resource-constrained environments.

This study addresses these gaps through a controlled three-model comparison and a qualitative semantic context analysis.

1 Introduction

1.1 Background and Motivation

Short Message Service (SMS) remains one of the most widely used communication channels, with billions of messages exchanged every day. This reach has made SMS an attractive vector for spam campaigns, phishing (smishing), and fraud. The consequences include financial loss through premium-rate scams, privacy breaches through credential theft and social engineering, security compromise through malicious links, erosion of user trust, and wasted network resources for carriers. Vulnerable groups, such as elderly users, tend to suffer disproportionate harm, and traditional defenses have proven inadequate against evolving spam tactics.

1.2 Problem Statement

Existing spam detection systems rely predominantly on three techniques. **Keyword-based filtering** is easily bypassed through character substitution (for example “FR33” instead of “FREE”), Unicode manipulation, and deliberate misspellings. **Rule-based heuristics** become obsolete as tactics evolve and require constant maintenance. **Shallow machine learning**, such as TF-IDF with Naive Bayes or Logistic Regression, treats text as a bag of independent words and therefore fails to capture semantic relationships between words, contextual variations in meaning, syntactic cues of malicious intent, and long-range dependencies within a message.

These limitations cause false negatives (missed spam), which compromise security, and false positives (blocked legitimate messages), which degrade user experience.

1.3 Research Objectives

1. Analyze SMS data to identify features that separate spam from legitimate messages beyond keyword frequency.
2. Implement two baseline classifiers: TF-IDF with Logistic Regression, and a word-embedding Bidirectional LSTM (BiLSTM).
3. Fine-tune a pretrained Transformer (DistilBERT) for binary spam classification using transfer learning.
4. Evaluate all models with accuracy, precision, recall, F1-score, ROC-AUC, confusion matrices, and edge-case studies.
5. Identify specific message types on which the Transformer succeeds where traditional approaches fail, and explain why.

1.4 Research Contributions

The study contributes an empirical comparison of three architectural paradigms on one benchmark with identical splits and metrics, a qualitative semantic analysis of messages on which the Transformer corrects the baseline’s errors, and a reproducible description of preprocessing, training, evaluation, and deployment.

1.5 Document Organization

The literature review above summarizes related work. Section 2 describes the dataset, Section 3 presents the exploratory analysis, Section 4 details preprocessing, Section 5 outlines the methodology, and Section 6 describes the model architectures. Section 7 explains the experimental setup, Section 8 reports the results, Section 9 analyzes semantic context, Section 10 discusses implications, and Section 11 concludes.

2 Dataset Description

2.1 Data Source and Characteristics

This study uses the SMS Spam Collection from the UCI Machine Learning Repository (Almeida & Hidalgo, 2011), a widely adopted benchmark for spam detection research.

- **Total instances:** 5,572 English SMS messages in tab-separated format.
- **Sources:** SMS Spam Corpus v.0.1, the Grumbletext website, the NUS SMS Corpus, and Caroline Tag's PhD research.
- **Annotation:** manual labeling by human annotators, with quality verification.
- **Collection period:** approximately 2011 to 2012.

2.2 Data Schema and Attributes

The dataset has two fields, summarized in Table 1.

Table 1. Dataset schema and field descriptions

Attribute	Data type	Description	Values
label	Categorical	Message classification	"ham" (legitimate) or "spam"
message	String	Raw SMS text	Variable length (2 to 910 characters)

2.3 Dataset Quality Assessment

The dataset is complete: it contains no missing values and no malformed records, and the few encoding issues were handled by UTF-8 normalization. The label distribution is imbalanced, with 4,825 ham messages (86.6%) and 747 spam messages (13.4%), a ratio of roughly 6.5 to 1. This reflects real-world conditions, where legitimate messages greatly outnumber spam, but it requires metrics such as F1-score and ROC-AUC rather than accuracy alone.

The spam class covers prize and lottery scams, premium-rate call solicitations, phishing attempts, adult content promotions, and financial fraud. The ham class covers personal conversations, appointment reminders, social coordination, and informational updates.

2.4 Dataset Limitations and Considerations

- **Temporal bias:** messages from 2011 to 2012 may not reflect contemporary tactics such as cryptocurrency scams or recent phishing campaigns, although the underlying linguistic patterns remain relevant.
- **Geographic bias:** the data is predominantly UK-based (British currency symbols and phone number formats), which may limit generalization to other regions and languages.
- **Size:** with 5,572 messages the dataset is small by deep learning standards; transfer learning from a pretrained Transformer partly mitigates this.
- **Binary labels:** the labels do not distinguish spam subtypes such as phishing versus advertising.

Despite these limitations, the collection remains the most widely used SMS spam benchmark and allows direct comparison with published work.

3 Exploratory Data Analysis

Exploratory data analysis (EDA) was conducted to understand the dataset, identify patterns that separate spam from legitimate messages, and guide preprocessing and modeling decisions.

3.1 Class Distribution Analysis

Figure 1 shows the class imbalance. A naive classifier that labels every message as ham would reach 86.6% accuracy while detecting no spam at all. The imbalance therefore has three methodological consequences:

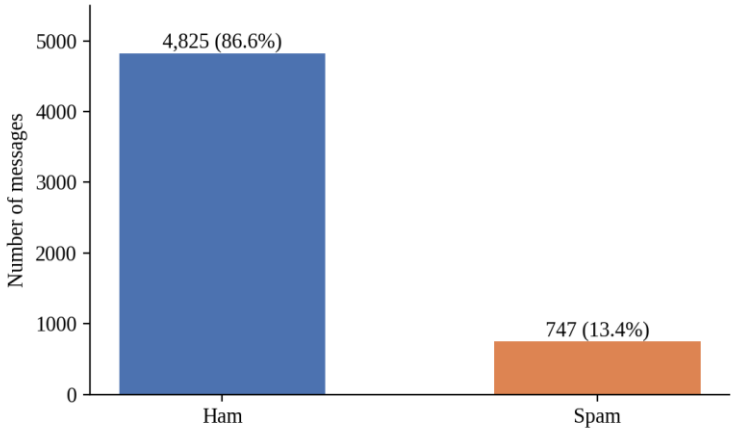


Figure 1. *Class distribution of the SMS messages (ham 86.6%, spam 13.4%)*

1. **Evaluation metrics:** precision, recall, F1-score, and ROC-AUC are prioritized over accuracy.
2. **Data splitting:** stratified sampling keeps the class proportions equal in the training, validation, and test sets.
3. **Model training:** class weighting or oversampling could reduce majority-class bias, but neither was applied in this study.

3.2 Message Length Statistics

Spam and ham messages differ markedly in length (Tables 2 and 3). Spam messages are longer on average (138.7 versus 71.5 characters) and far more uniform (standard deviation of 28.9 versus 58.4), which suggests templated generation. Ham messages include extreme outliers of up to 910 characters, whereas spam rarely exceeds 223 characters. The minimum spam length is also higher (13 versus 2 characters), because spam needs enough content to convey its offer.

Table 2. Character length statistics by class

Statistic	Ham	Spam	Difference
Mean	71.5	138.7	+94.0%
Standard deviation	58.4	28.9	−50.5%
Minimum	2	13	+550%
25th percentile	33	133	+303%

Statistic	Ham	Spam	Difference
Median	52	149	+187%
75th percentile	93	157	+69%
Maximum	910	223	−75.5%

Table 3. Word count statistics by class

Statistic	Ham	Spam	Difference
Mean	14.3	23.9	+67.1%
Standard deviation	11.5	5.8	−49.6%
Minimum	1	2	+100%
Median	11	25	+127%
Maximum	171	35	−79.5%

Length is therefore a useful but insufficient feature: it separates the classes well on average but cannot detect short or carefully disguised spam on its own.

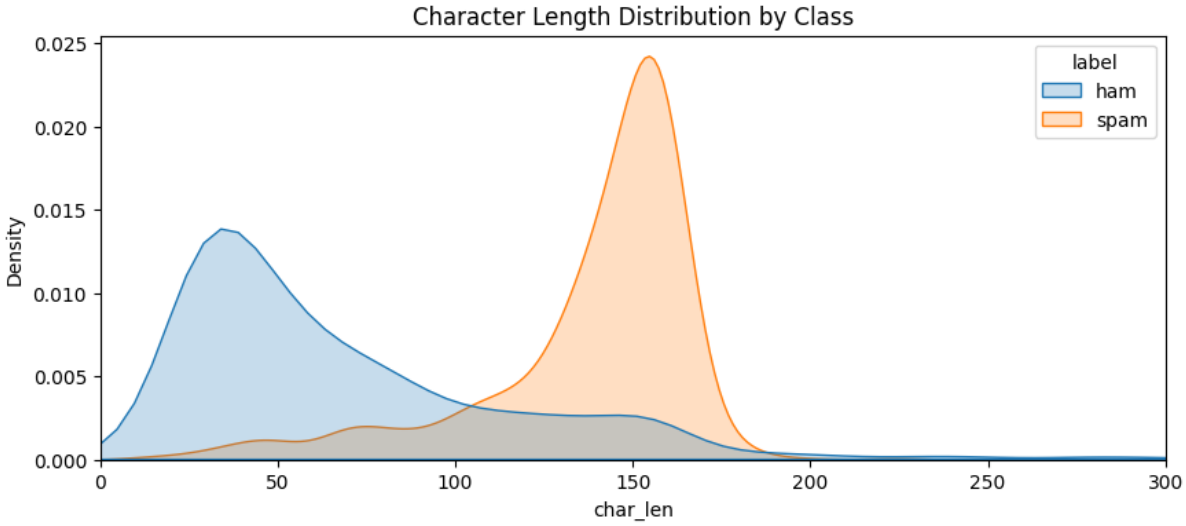


Figure 2. *Character length distribution by class*

3.3 Vocabulary and Semantic Patterns

Word frequency analysis (word clouds) highlighted the vocabulary that distinguishes the two classes. Table 4 groups the most prominent terms by theme.

Table 4. Dominant vocabulary themes by class

Class	Theme	Representative terms
Ham	Personal and relational	know, love, call, friend, want, need
Ham	Temporal references	time, day, now, later, today, tomorrow
Ham	Conversational	u, ur, yeah, ok, good, great
Ham	Actions and context	go, come, get, give, take, home, work, back
Spam	Promotional triggers	FREE (most prominent), call, text, reply
Spam	Prize and incentive	claim, win, prize, award, won
Spam	Urgency and imperatives	now, URGENT, STOP, txt
Spam	Monetary and contact cues	150p, mobile, message, contact

Ham vocabulary is diverse and conversational, with personal pronouns, informal abbreviations, and context-specific references. Spam vocabulary concentrates on four themes: urgency ("now", "urgent"), calls to action ("call", "text", "claim"), value propositions ("free", "win", "prize"), and contact mechanisms (premium-rate numbers and reply codes). The word "FREE" dominates the spam vocabulary, but relying on keyword presence alone is vulnerable to obfuscation.

3.4 Implications for Feature Engineering

The analysis identified three groups of discriminative features: length-based features (character and word counts, and lower length variance for spam); vocabulary features (promotional, urgency, contact, and monetary terms); and structural features (all-capital words, punctuation density, and the presence of URLs or phone numbers).

Sophisticated attacks defeat such surface features through keyword obfuscation ("FR33", "W!N"), semantic disguise (spam intent embedded in a conversational narrative), and context manipulation (legitimate-looking language delivering a phishing lure). This motivates semantic context evaluation beyond surface-level features.

4 Data Preprocessing

Preprocessing converts raw, noisy text into clean inputs for machine learning while preserving the semantic content needed for classification.

4.1 Text Cleaning and Normalization

A cleaning pipeline was applied to the TF-IDF and BiLSTM inputs:

```
import re

def clean_text(text):
    text = text.lower()                                  # 1
    text = re.sub(r'http\S+|www\S+|https\S+', '<URL>', text)    # 2
    text = re.sub(r'\d+', '<NUMBER>', text)                  # 3
    text = re.sub(r'[^\w\s]', '', text)                      # 4
    text = re.sub(r'\s+', ' ', text).strip()                 # 5
    return text
```

1. **Lowercasing** treats “FREE”, “Free”, and “free” identically and reduces vocabulary size, at the cost of losing capitalization as a spam signal.
2. **URL normalization** replaces every URL with one token, because the specific address matters less than the presence of a link.
3. **Number normalization** replaces numeric sequences (phone numbers, prices) with a single token, preserving the fact that a number is present.
4. **Punctuation removal** strips special characters. Because it runs after the substitutions, the placeholders become the plain tokens NUMBER and URL.
5. **Whitespace normalization** collapses repeated spaces for consistent tokenization.

For example, the message “WINNER!! As a valued customer you have been selected to receive a £900 prize reward! Call 09061234567 NOW! Visit http://claim-prize.com” becomes “winner as a valued customer you have been selected to receive a NUMBER prize reward call NUMBER now visit URL”. The Transformer inputs were not cleaned this way: they keep the original casing, punctuation, and numbers, because the pretrained tokenizer and model use that information.

4.2 Label Encoding

Labels were encoded as integers for compatibility with binary cross-entropy loss: ham became 0 (negative class) and spam became 1 (positive class).

4.3 Dataset Splitting Strategy

The dataset was divided into training, validation, and test sets using stratified sampling with a fixed random seed of 42 (Table 5). The first split separated 70% for training, and the remaining 30% was divided equally into validation and test sets.

Table 5. Stratified dataset partitions

Subset	Proportion	Messages	Purpose
Training	70%	3,900	Learning model parameters
Validation	15%	836	Monitoring and model selection
Test	15%	836	Final unbiased evaluation

Stratification keeps the spam share at about 13.4% in every subset. Without it, a random split could produce a test set with a very different spam ratio and unreliable results. The test set (836 messages) contains 724 ham and 112 spam messages and was never used during training. Vocabulary construction and TF-IDF fitting used the training set only, which prevents data leakage.

4.4 Model-Specific Preprocessing

- **TF-IDF with Logistic Regression:** cleaned text is converted to a sparse matrix with a 3,000-term vocabulary and English stop-word removal.
- **BiLSTM:** a vocabulary of the 5,000 most frequent training tokens (including <PAD> and <UNK>) maps text to integer sequences, padded or truncated to 50 tokens.
- **DistilBERT:** the pretrained WordPiece tokenizer (30,522 subword vocabulary) is applied to the raw text with truncation and padding to 128 tokens. No stop-word removal is performed, and punctuation is retained.

Transformers need less aggressive preprocessing than traditional models because subword tokenization handles capitalization, punctuation, and rare words directly.

5 Methodology

This study uses a controlled comparative design that evaluates three classification paradigms representing the evolution of natural language processing for spam detection.

5.1 Research Framework

All models share the same dataset, the same stratified 70/15/15 split, the same evaluation metrics, and the same random seed (42). The independent variable is the model architecture and its input representation (sparse TF-IDF vectors, learned dense embeddings, or contextual subword embeddings). Preprocessing was standardized per model type, and every model was evaluated on the identical held-out test set. Two hypotheses were tested:

- **H0:** TF-IDF with Logistic Regression performs equivalently to Transformer-based semantic evaluation for SMS spam detection.
- **H1:** The Transformer achieves clearly higher F1-score and recall through semantic context understanding, particularly on obfuscated and context-dependent spam.

5.2 Model Selection Rationale

The three architectures (Table 6) represent distinct modeling paradigms.

Table 6. Comparison of the selected model paradigms

Model	Representation	Strengths	Limitations
TF-IDF + Logistic Regression	Sparse bag-of-words with TF-IDF weights	Very fast; interpretable coefficients; low compute; strong keyword baseline	No semantics or word order; vulnerable to obfuscation
Embedding + BiLSTM	Learned dense word embeddings	Captures word order and bidirectional context; learns distributed representations	Limited long-range modeling; sequential processing; needs more data
Fine-tuned DistilBERT	Contextual subword embeddings	Self-attention over the whole message; transfer learning; robust to rare words	Higher compute cost; less interpretable; 66M parameters

TF-IDF with Logistic Regression represents current industry practice and provides the performance floor. The BiLSTM represents deep learning before Transformers. DistilBERT was chosen over full BERT because it has 40% fewer parameters (66 million versus 110 million), roughly 60% faster inference, and retains about 97% of BERT's language understanding capability, which is a favorable balance of accuracy and efficiency for real-time, mobile-oriented deployment.

SMS text is short, informal, and full of slang, deliberate misspellings, character obfuscation, numbers, and deceptive links. Self-attention lets DistilBERT weigh all tokens of a message

simultaneously, so it can relate, for example, a premium-rate number to a buried cost disclaimer regardless of their positions or of character-level obfuscation.

5.3 Evaluation Metrics

Because of the class imbalance (86.6% ham, 13.4% spam), several complementary metrics were used. TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives (Table 7), where spam is the positive class.

Table 7. Confusion matrix definition

	Predicted ham (0)	**Predicted spam (1)**
Actual ham (0)	True negative (TN)	False positive (FP): legitimate message blocked
Actual spam (1)	False negative (FN): spam missed	True positive (TP)

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) \quad (1)$$

$$Precision = TP / (TP + FP) \quad (2)$$

$$Recall = TP / (TP + FN) \quad (3)$$

$$F1 = 2 \times Precision \times Recall / (Precision + Recall) \quad (4)$$

- **Accuracy** is the overall share of correct predictions. It is a secondary metric because it is misleading under class imbalance.
- **Precision** is the share of predicted spam that is truly spam. Low precision means legitimate messages are quarantined.
- **Recall** is the share of actual spam that is detected. In security applications a missed threat is typically costlier than a false alarm, so recall is the most critical metric.
- **F1-score** is the harmonic mean of precision and recall and serves as the primary comparison metric.
- **ROC-AUC** measures discrimination across all decision thresholds, from 0.5 (random guessing) to 1.0 (perfect separation).

5.4 Training Procedures

- **TF-IDF with Logistic Regression:** L2-regularized (C = 1.0) model trained with the L-BFGS solver for up to 1,000 iterations. Training takes under one second.
- **BiLSTM:** Adam optimizer (learning rate 0.001), binary cross-entropy loss, batch size 32, 5 epochs. Training takes about 2 minutes on a GPU.
- **DistilBERT:** AdamW optimizer (learning rate 2×10^{-5}, weight decay 0.01), batch size 16, 3 epochs using the Hugging Face Trainer. Training takes about 2.5 minutes on a GPU.

A short fine-tuning schedule is appropriate because the pretrained weights already encode general language knowledge, and the small training set (3,900 messages) risks overfitting if

training is prolonged. Hyperparameters followed standard recommended settings; no extensive tuning was performed.

6 Model Architectures

6.1 Baseline Model: TF-IDF with Logistic Regression

This baseline represents traditional keyword-based filtering. TF-IDF converts each message into a weighted vector in which terms that are frequent in a message but rare across the corpus receive high scores:

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t) \quad (5)$$

$$IDF(t) = log(N / n_t) \quad (6)$$

Here TF(t, d) is the relative frequency of term t in message d, N is the number of messages, and n_t is the number of messages containing t. The vectorizer keeps the 3,000 most important unigrams after English stop-word removal, giving a sparse matrix of 3,900 samples by 3,000 features for the training set.

Logistic Regression then estimates the probability of spam with a sigmoid function, where x is the TF-IDF vector, w the learned weights, and b the bias:

$$P(y = 1 \mid x) = \sigma(w^T x + b) = 1 / (1 + e^{\wedge}{-}(w^T x + b)) \quad (7)$$

A message is classified as spam when this probability is at least 0.5. The model is fast (a single matrix-vector product per message) and interpretable, since each weight shows how strongly a term indicates spam or ham. Its limitations are a linear decision boundary, no word order or context, and no semantic link between variants such as “FREE” and “FR33”.

6.2 Intermediate Model: Embedding and Bidirectional LSTM

The BiLSTM classifier learns word embeddings and models sequential dependencies in both directions. Its pipeline has four stages:

1. **Vocabulary and sequences:** the 5,000 most frequent training tokens are mapped to integers (0 for <PAD>, 1 for <UNK>), and each message is truncated or padded to 50 tokens.
2. **Embedding layer:** each token becomes a learned 64-dimensional vector (in the spirit of word embeddings such as Word2Vec and GloVe; Mikolov et al., 2013; Pennington et al., 2014), with a zero vector for padding.
3. **Bidirectional LSTM layer:** one layer with a hidden size of 64 reads the sequence forward and backward. The final forward and backward hidden states are concatenated into a 128-dimensional representation.
4. **Classification head:** dropout (p = 0.3), a linear layer from 128 to 1 unit, and a sigmoid activation that outputs the spam probability.

The model has approximately 0.39 million trainable parameters, of which about 0.32 million are in the embedding table. Training loss decreased from 0.3394 in epoch 1 to 0.0587 in epoch 5. Its sequential processing limits long-range modeling and prevents parallelization.

6.3 Proposed Model: Fine-Tuned DistilBERT

DistilBERT is a distilled version of BERT with 6 Transformer encoder layers, 12 attention heads, a hidden size of 768, and 66 million parameters (Table 8). It was pretrained on English Wikipedia and BooksCorpus (about 3.3 billion words) using masked language modeling together with a distillation objective, and was initialized here from the distilbert-base-uncased checkpoint.

Table 8. DistilBERT model specifications

Parameter	Value
Encoder layers	6
Hidden size	768
Attention heads	12 per layer
Vocabulary	30,522 WordPiece tokens
Maximum sequence length	128 tokens
Total parameters	66 million
Fine-tuning	3 epochs, learning rate 2×10^{-5}

Tokenization. WordPiece splits words into subword units, which makes the model robust to misspellings and unseen words. For example, "URGENT! Your mobile number has won a £900 prize" is tokenized as [CLS] urgent ! your mobile number has won a £ 900 prize [SEP], and an obfuscated word such as "FR33" is decomposed into subwords instead of being treated as unknown.

Self-attention. Each encoder layer computes relevance weights between all token pairs from query (Q), key (K), and value (V) matrices, where d_k = 64 is the key dimension. Twelve parallel heads let the model attend to different relationships at once:

$$Attention(Q, K, V) = softmax(QK^T / \sqrt{d_k})\, V \qquad (8)$$

This allows a word such as "prize" to be interpreted in light of nearby words such as "FREE", "calling", and "now", so that the same word is represented differently in "feel free to ask" and "free prize".

Classification head. The representation of the [CLS] token, which aggregates information from the whole sequence through self-attention, is passed through the classification layers of

the Hugging Face sequence classification model to produce two logits. A softmax converts them into [P(ham), P(spam)]. Figure 2 summarizes the pipeline.

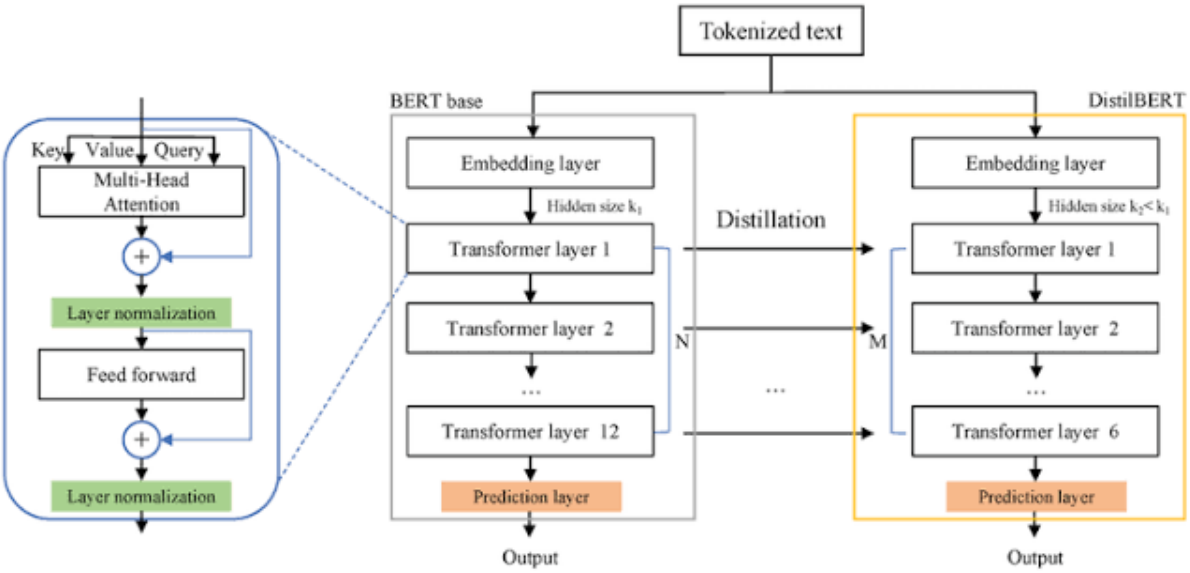


Figure 2. DistilBERT architecture and inference pipeline

Unlike the BiLSTM, DistilBERT attends to all positions simultaneously and processes tokens in parallel. The trained model occupies roughly 260 MB, and inference takes tens of milliseconds per message on a GPU (latency was not formally benchmarked).

7 Experimental Setup

7.1 Hardware and Software Environment

Experiments ran on Google Colab with an NVIDIA Tesla T4 GPU (16 GB VRAM), a 2-core Intel Xeon CPU, 12 GB of system memory, and 100 GB of disk storage. The software stack comprised Ubuntu 20.04 LTS, Python 3.8.10, PyTorch 2.0.1, Hugging Face Transformers 4.30.2, scikit-learn 1.2.2, NumPy 1.23.5, Pandas 1.5.3, and CUDA Toolkit 11.8. The models were implemented with PyTorch (Paszke et al., 2019), scikit-learn (Pedregosa et al., 2011), and Hugging Face Transformers (Wolf et al., 2020). All random number generators (Python, NumPy, and PyTorch for CPU and GPU) were seeded with 42 for reproducibility.

7.2 Training Configuration

The TF-IDF vectorizer used 3,000 features, English stop words, and unigrams only. Logistic Regression used the L-BFGS solver with C = 1.0 and up to 1,000 iterations. The BiLSTM used the Adam optimizer with a learning rate of 0.001 and weight decay of 1×10^{-5}, binary cross-entropy loss, a batch size of 32, and 5 epochs. DistilBERT was fine-tuned with the following configuration:

```
training_args = TrainingArguments(
    output_dir="./results_transformer",
    num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    learning_rate=2e-5,
    weight_decay=0.01,
    logging_steps=50,
    save_strategy="epoch",
    seed=42,
    report_to="none")
```

Figure 3 shows the training loss logged every 50 steps. The loss fell sharply from 0.2598 at step 50 to 0.0480 at step 100, then continued to decrease with minor fluctuations, reaching 0.0161 at step 700. The overall downward trend indicates stable convergence.

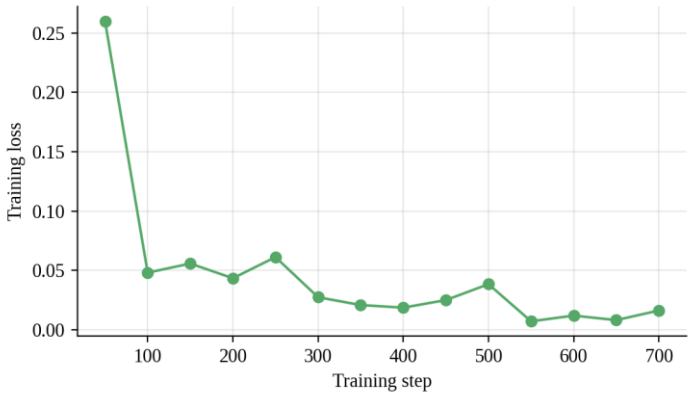


Figure 3. DistilBERT training loss over 700 steps

7.3 Evaluation Protocol

All three models were evaluated on the same held-out test set of 836 messages (15% of the data). Predicted probabilities were thresholded at 0.5 to obtain class labels. For each model, accuracy, precision, recall, F1-score, and ROC-AUC were computed with scikit-learn, with spam as the positive class, and confusion matrices were derived from the predictions.

8 Results and Performance Analysis

8.1 Overall Performance Comparison

Table 9 and Figure 4 compare the three models on the test set.

Table 9. Performance metrics on the test set (N = 836)

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
TF-IDF + Logistic Regression	0.9737	0.9592	0.8393	0.8952	0.9896
Embedding + BiLSTM	0.9773	0.9697	0.8571	0.9100	0.9858
DistilBERT (Transformer)	**0.9928**	**0.9732**	**0.9732**	**0.9732**	**0.9990**

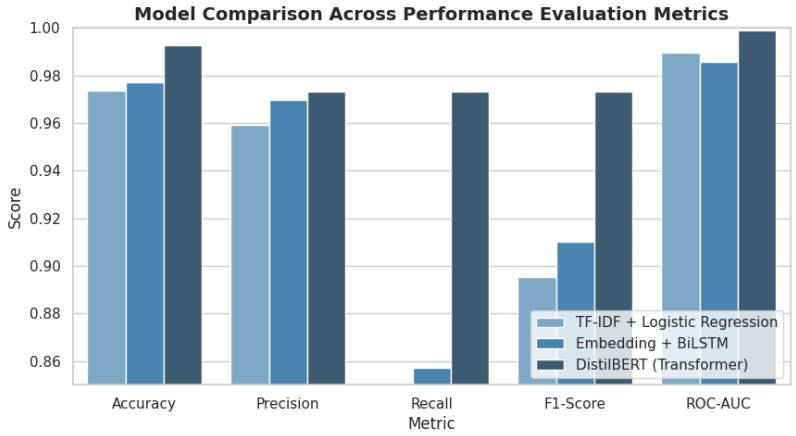


Figure 4. Comparison of the three models across evaluation metrics

- **Accuracy:** all models exceed 97%. DistilBERT reaches 99.28%, an improvement of 1.91 percentage points over TF-IDF.
- **Precision:** all models exceed 95% (DistilBERT 97.32%, BiLSTM 96.97%, TF-IDF 95.92%), so false alarms are rare across all approaches.
- **Recall (most critical):** this shows the largest gap. DistilBERT detects 109 of 112 spam messages (97.32%), the BiLSTM 96 (85.71%), and TF-IDF 94 (83.93%). DistilBERT improves recall by 11.61 percentage points over the BiLSTM and 13.39 over TF-IDF.
- **F1-score:** 97.32% for DistilBERT versus 91.00% for the BiLSTM and 89.52% for TF-IDF.
- **ROC-AUC:** DistilBERT achieves 0.9990, compared with 0.9896 (TF-IDF) and 0.9858 (BiLSTM), indicating near-perfect separation of the classes.

To illustrate the practical effect, consider a carrier filtering 100,000 messages per day at a 13.4% spam rate (13,400 spam messages). Applying each model's false negative rate would leave about 2,153 spam messages undetected per day for TF-IDF, 1,915 for the BiLSTM, and 359 for DistilBERT, so DistilBERT would block roughly 1,794 more threats per day than the TF-

IDF baseline. This is an extrapolation from the test set and assumes the same spam share and message mix.

8.2 Confusion Matrix Analysis

Figure 5 shows the confusion matrices. The test set contains 724 ham and 112 spam messages.

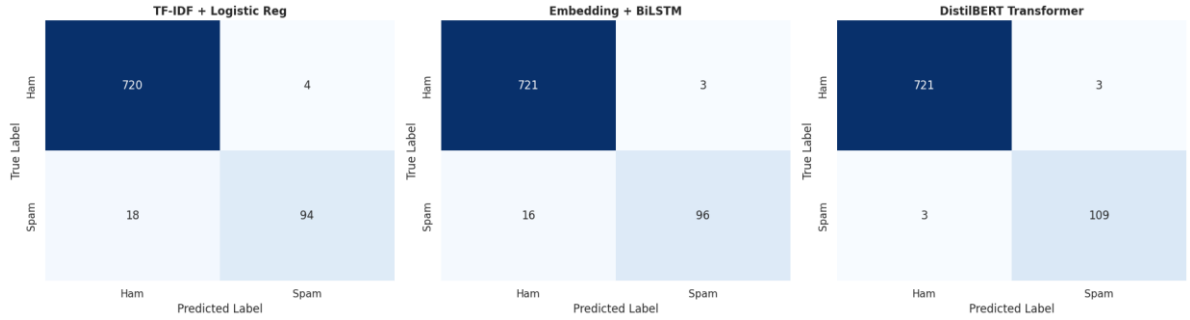


Figure 5. Confusion matrices for the three models on the test set

- **TF-IDF:** TN = 720, FP = 4, FN = 18, TP = 94. The false positive rate is 0.55% and the false negative rate is 16.07%.
- **BiLSTM:** TN = 721, FP = 3, FN = 16, TP = 96. The false positive rate is 0.41% and the false negative rate is 14.29%.
- **DistilBERT:** TN = 721, FP = 3, FN = 3, TP = 109. The false positive rate is 0.41% and the false negative rate is 2.68%.

Table 10 summarizes the error counts relative to the TF-IDF baseline.

Table 10. Error summary relative to the TF-IDF baseline

Metric	TF-IDF	BiLSTM	DistilBERT	Change vs. TF-IDF
False negatives (missed spam)	18	16	3	−83.3%
False positives (false alarms)	4	3	3	−25.0%
True positives (detected spam)	94	96	109	+16.0%

8.3 ROC-AUC Analysis

DistilBERT obtained the highest ROC-AUC (0.9990), ahead of TF-IDF with Logistic Regression (0.9896) and the BiLSTM (0.9858); random guessing corresponds to 0.5. Because AUC is threshold-independent, the result shows that DistilBERT ranks spam above ham more reliably across all possible decision thresholds, not only at the default threshold of 0.5. The baselines also have high AUC values, which shows that their weakness lies mainly in the recall they achieve at the default threshold rather than in an inability to rank messages at all.

9 Semantic Context Evaluation

Beyond quantitative metrics, a qualitative analysis examined how the Transformer's semantic understanding produces its advantage.

9.1 Error Analysis Methodology

The analysis selected test messages that the TF-IDF baseline classified incorrectly and DistilBERT classified correctly ("contextual recovery" cases). This yielded 19 messages, or 2.3% of the test set, comprising 16 spam messages recovered from false negatives and 3 legitimate messages recovered from false positives. The case studies below are representative examples. The explanations of why each model succeeded or failed are interpretations based on the message content and each model's design, not measured attention values.

9.2 Case Study 1: Obfuscated Phishing Without Spam Keywords

Message: "This msg is for your mobile content order It has been resent as previous attempt failed due to network error Queries to customersqueries@netvision.uk.com"

Ground truth: spam. **TF-IDF:** ham (incorrect). **DistilBERT:** spam (correct).

The message contains none of the typical spam keywords such as "free", "win", or "prize". Each of its individual terms carries only a weak spam signal, so the combined TF-IDF score stays below the decision threshold. DistilBERT can relate the unsolicited "order", the claim of a failed delivery attempt, and the unfamiliar contact address as parts of a familiar phishing structure: a false premise, a manufactured problem, and a contact point.

9.3 Case Study 2: Social Engineering with Conversational Disguise

Message: "CLAIRE here am havin borin time & am now alone U wanna cum over 2nite? Chat now 09099725823 hope 2 C U Luv CLAIRE xx Calls£1/minmoremobsEMSPOBox45PO139WA"

Ground truth: spam. **TF-IDF:** ham (incorrect). **DistilBERT:** spam (correct).

The informal wording closely resembles legitimate personal messages. DistilBERT reads the whole sequence and can connect the premium-rate-style number to the buried charge notice ("Calls£1/min") and notice the mismatch between an intimate personal invitation and a commercial chat-line contact number.

9.4 Case Study 3: Explicit Spam Missed by TF-IDF

Message: "Want explicit SEX in 30 secs? Ring 02073162414 now! Costs 20p/min"

Ground truth: spam. **TF-IDF:** ham (incorrect). **DistilBERT:** spam (correct).

The unigram TF-IDF model split the phrase into individual words with moderate weights and had no stronger evidence. DistilBERT's pretrained representations cover adult-content terminology, and the model links it with time pressure ("in 30 secs") and a per-minute call charge ("Costs 20p/min").

9.5 Case Study 4: Legitimate Message Flagged by the Baseline

Message: “U 2.”

Ground truth: ham. **TF-IDF:** spam (false alarm). **DistilBERT:** ham (correct).

After cleaning and stop-word removal, almost no informative features remain in a two-character message, so the TF-IDF prediction is unreliable. DistilBERT recognizes “U 2” as common informal shorthand for “you too” with no commercial intent.

9.6 Case Study 5: Legitimate Message with Unusual Vocabulary

Message: “Funny fact Nobody teaches volcanoes 2 erupt, tsunamis 2 arise, hurricanes 2 sway aroundn no 1 teaches hw 2 choose a wife Natural disasters just happens”

Ground truth: ham. **TF-IDF:** spam (false alarm). **DistilBERT:** ham (correct).

Rare words such as “volcanoes”, “tsunamis”, and “hurricanes” receive high inverse document frequency weights, and the message length is close to the typical spam length, both of which distort the TF-IDF decision. DistilBERT’s pretraining on Wikipedia and books exposes it to analogies and humor, so it can treat the “Funny fact” framing as non-commercial.

9.7 Summary of Semantic Patterns

- **Obfuscation resistance:** spam intent is detected even without traditional keywords.
- **Contextual coherence:** the entire message narrative is evaluated instead of isolated terms.
- **Structural pattern recognition:** buried disclosures and typical spam structures are recognized.
- **Intent understanding:** informational or playful content is distinguished from solicitation.
- **Transfer learning robustness:** subword tokenization and pretraining handle rare words, slang, and misspellings.

10 Discussion

10.1 Interpretation of Results

The results support the alternative hypothesis H1. DistilBERT improved recall by 11.61 percentage points over the BiLSTM and reduced missed spam from 18 to 3 messages relative to TF-IDF, while maintaining a false positive rate of 0.41%. The case studies suggest that self-attention helps by relating distant parts of a message, such as a phone number and its cost disclaimer. The study used a single stratified split without repeated runs or a statistical significance test, so the difference should be read as strong evidence on this dataset rather than a formally tested guarantee. With only 112 spam messages in the test set, each missed message changes recall by about 0.9 percentage points.

10.2 Comparison with Related Work

Earlier studies on the UCI SMS Spam Collection report accuracies of roughly 96% to 98% (Almeida & Hidalgo, 2011; Gupta et al., 2018; Roy et al., 2020). The 99.28% accuracy obtained here is higher, but the studies differ in splits, preprocessing, and metrics, so the figures are only indicative. The controlled comparison in this study is the more informative one.

10.3 Practical Implications for Deployment

A carrier-side deployment would intercept each message at the gateway, run DistilBERT inference, and quarantine messages whose spam probability exceeds 0.5, delivering all others to the inbox. Because CPU inference is considerably slower than for TF-IDF, a GPU or an optimized runtime (for example quantization or ONNX export) is advisable at scale, and the decision threshold can be tuned to balance missed spam against false alarms.

10.4 Limitations and Challenges

1. **Dataset size and age:** 5,572 English messages from 2011 to 2012 limit coverage of modern spam. Transfer learning mitigates the size but not the age.
2. **Temporal drift:** spam tactics evolve, so periodic retraining on fresh data is required.
3. **Computational cost:** Transformer inference is more expensive than TF-IDF.
4. **Language coverage:** the model is English-only and would need multilingual pretraining for other languages.
5. **Interpretability:** decisions are less transparent than those of a linear model.
6. **Adversarial robustness and evaluation scope:** character-level perturbations may still degrade performance, and results come from one split and one run without significance testing.

10.5 Future Research Directions

- Evaluation on larger, more recent, and multilingual datasets, with multilingual models such as mBERT or XLM-RoBERTa.
- Quantization and on-device deployment of lightweight models, and comparison with RoBERTa, DeBERTa, or MobileBERT.

- Ensembles, explainable AI integration, multimodal detection (images, text, and sender metadata), and few-shot or federated approaches.

11 Conclusion

11.1 Summary of Findings

This study compared TF-IDF with Logistic Regression, an embedding-based BiLSTM, and a fine-tuned DistilBERT model for mobile SMS spam detection. DistilBERT achieved 99.28% accuracy, 97.32% precision, 97.32% recall, a 97.32% F1-score, and a 0.9990 ROC-AUC, and it reduced missed spam by 83% relative to the TF-IDF baseline (18 to 3 messages). Case studies showed that it recognizes obfuscated phishing, conversational social engineering, and unusual but legitimate language that keyword-based features misjudge.

11.2 Research Contributions

- **Theoretical:** empirical evidence that contextual semantic models outperform keyword-based and sequential models on this benchmark, and a qualitative framework of five semantic mechanisms that explain the gains.
- **Practical:** a reproducible training and evaluation pipeline and a deployment outline for carrier-level filtering.

11.3 Implications for Mobile Security

Adaptive, AI-based filtering allows mobile carriers and security vendors to move beyond brittle rule-based systems and better protect vulnerable users from smishing and financial fraud.

11.4 Lessons Learned

Pretraining is valuable even with a small labeled dataset; context matters more than individual keywords; the accuracy–efficiency trade-off can be managed with a distilled model such as DistilBERT; and qualitative error analysis is essential for understanding classifier behavior beyond summary metrics.

11.5 Final Recommendations

- **For researchers:** evaluate multilingual models and multimodal fusion of text and metadata on larger, more recent corpora, using repeated runs and significance tests.
- **For practitioners:** consider DistilBERT-based filtering in carrier infrastructure with periodic retraining and a tunable decision threshold.
- **For policymakers:** support the creation of contemporary, openly available spam corpora and encourage semantic filtering standards.

References

Almeida, T., & Hidalgo, J. G. (2011). *SMS Spam Collection* [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5CC84

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186. (arXiv:1810.04805, 2018)

Gupta, V., Mehta, A., Goel, A., Dixit, U., & Pandey, A. C. (2018). Spam detection using ensemble learning. *International Conference on Computational Intelligence and Data Science (ICCIDS 2018)*.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780.

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems, 32*, 8024–8035.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. *Proceedings of EMNLP 2014*, 1532–1543.

Roy, P. K., Singh, J. P., & Banerjee, S. (2020). Deep learning to filter SMS spam. *Future Generation Computer Systems*, *102*, 524–533.

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, *30*, 5998–6008.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., … Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. *Proceedings of EMNLP 2020: System Demonstrations*, 38–45.

Appendix A: Hyperparameter Configuration

Table A1 lists the hyperparameters used for each model.

Table A1. Hyperparameter settings across the evaluated models

Model	Hyperparameter	Value
TF-IDF + LogReg	Max features / stop words	3,000 / English
	N-gram range	(1, 1)
	Max iterations / solver	1,000 / L-BFGS
	Regularization (C)	1.0
BiLSTM	Vocabulary size	5,000
	Embedding / hidden dimension	64 / 64
	Layers / dropout	1 / 0.3
	Learning rate / weight decay	$0.001 / 1 \times 10^{-5}$
	Batch size / epochs	32 / 5
DistilBERT	Pretrained model	distilbert-base-uncased
	Max sequence length	128
	Learning rate / weight decay	$2 \times 10^{-5} / 0.01$
	Batch size / epochs	16 / 3
	Warmup steps	0

Appendix B: Sample Predictions

Table A2 shows predictions for representative test messages. A check mark (✓) denotes a correct prediction and a cross (✗) an incorrect one.

Table A2. Sample correct and incorrect predictions across models

SMS text (truncated)	True label	TF-IDF	BiLSTM	DistilBERT
"Hey mate, are we still…"	HAM	HAM ✓	HAM ✓	HAM ✓
"URGENT! Your mobile number has won…"	SPAM	SPAM ✓	SPAM ✓	SPAM ✓
"This msg is for your mobile content…"	SPAM	HAM ✗	HAM ✗	SPAM ✓
"CLAIRE here am havin borin time…"	SPAM	HAM ✗	HAM ✗	SPAM ✓
"U 2."	HAM	SPAM ✗	HAM ✓	HAM ✓